Série 2 - Complexité des algorithmes

Exercice 1

En utilisant les propriétés, déterminez la complexité asymptotique en notation Grand-Oh pour les complexités temporelles suivantes, où k est une constante entière avec k << n :

1- $T(n) = 4n^2 + 30n + 2000$

2- $T(n) = 500n^3 + 10n^2 + 70n + 13$

3- $T(n) = 2^n + 17n^2 + 20n + 7$

4- $T(n) = 40k + 3$

5- $T(n) = 3log_2(n) + 2n + 4$

6- $T(n) = 2log(n) + 3$

7- $T(n) = 4log_2(k) + 2kn^2 + 1$

Exercice 2

Considérons deux algorithmes A1 et A2, et posons T1(n) et T2(n) leurs complexités temporelles respectives données par :

$T_1(n) = 5n^3 + 17n^2 + 10n + 13$ $T_2(n) = 9n^2 + 50n + 10$

1- Déterminez la complexité asymptotique des deux algorithmes en termes de notation Grand-Oh. Quel algorithme a la meilleure complexité asymptotique ?

2- Montrez que vos solutions sont correctes à l'aide de la définition :

$$T(n) = O\big(f(n)\big),\ \text{Si}\ \exists\, c > 0,\ \exists\, n_0 \geq 0,\ \text{tel que}\ \forall n \geq n_0,\ T(n) \leq cf(n)$$

3- Soit l'algorithme suivant **A** :

Begin

If (cond) **Then** A1 **Else** A2 **Endif** ;

End.

Quelle est la complexité asymptotique de l'algorithme **A** dans :

- Le meilleur cas. - Le pire des cas. Justifiez vos réponses.

Exercice 3

Les algorithmes A et B prennent exactement TA(n) = 0,1n^2 log10(n) et TB(n) = 2,5n^2 microsecondes, respectivement, pour un problème de taille **n**.

1- Choisissez l'algorithme, qui est le meilleur dans le sens Grand-Oh, et trouvez une taille de problème n0 telle que pour toute taille plus grande n > n_0, l'algorithme choisi surpasse l'autre.

2- Si vos problèmes sont de taille n ≤ 10^9, quel algorithme recommanderiez-vous d'utiliser ?

Exercice 4

Déterminez la complexité temporelle des actions suivantes (en fonction du nombre d'itérations effectuées), où *m* et *n* sont deux entiers positifs.

Action A

```
i←1 ; j←1 ;
While i≤n And j≤m Do
    i←i+1 ; j←j+1 ;
Endwhile ;
```

Action B

```
i←1 ; j←1 ;
While i≤n Or j≤m Do
      i←i+1 ; j←j+1 ;
Endwhile ;
```

Action C

```
i←1 ; j←1 ;
While i≤n   Do
        If j≤m   Then j←j+1
                 Else i←i+1
        Endif;
Endwhile ;
```

Action D

```
i←1 ; j←1 ;
While i≤n Do
        If j≤m   Then j←j+1
                 Else i←i+1; j←1
        Endif;
Endwhile ;
```

Exercice 5

Supposons que deux algorithmes calculent la somme *Sn* définie par : $Sn = 1! + 2! + 3! + \cdots + n!$

Fournissez la complexité temporelle en fonction de n, puis donnez la complexité asymptotique de chaque algorithme en termes de notation Grand-Oh. Quel est le meilleur algorithme ?

```
Algorithm SFac1;
Var S,F, i, n : integer ;
Begin
        Read(n);
        F ←1 ; S ←0 ;
        For i←1 To n  Do
              F ← F * i ;
              S ← S + F ;
        Endfor ;
        Write(S) ;
End.
```

```
Algorithm SFac2;
Var S,F, i,j, n : integer ;
Begin
        Read(n);  S ←0 ;
        For i←1 To n  Do
                F←1 ;
                For j←1 To i  Do
                   F ← F * j ; Endfor ;
                S ← S + F ;
        Endfor ;
        Write(F) ;
End.
```

Exercice 6

Envisagez les actions suivantes :

```
Action A :
i← 0 ;
While I < n
Do      For j ← 1 To 10
        Do
            Write(i+1);
        Endfor;
        i ← i + 3;
Endwhile;
```

```
Action B :
i←1 ; k←1 ;
While i*i ≤ n
Do      For j ← 1 To i
        Do Write("Hello",k);
             k ← k + 1 ;
        Endfor;
        i ← i + 1;
Endwhile;
```

```
Action C :
i←0 ;
While i < n
Do      j ← 1;
        While j < n
        Do j ←2*j;
        Endwhile;
        i ← i + 1;
Endwhile;
```

Fournissez la complexité asymptotique en notation Grand-O pour chacun d'eux.

Exercice 7

Considérons un polynôme $P(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0$, où les coefficients n+1 (de 0 à n) sont stockés dans un tableau T.

1- Ecrire une fonction **ValPoly** permettant de calculer la valeur du polynôme en un point x.

2- Le mathématicien anglais **Horner** a proposé la formule suivante pour calculer la valeur du polynôme en un point x : $P(x) = (((a_n x + a_{n-1})x + a_{n-2})x + \cdots + a_1)x + a_0$.

Écrivez une **version Horner** de la fonction **ValPoly**.

3- Comparez la complexité des deux fonctions.

Exercice 8

Écrivez une fonction **SumDigits** qui renvoie la somme des chiffres d'un entier N donné, puis indiquez sa complexité asymptotique.

Exercice 9

Soit T un tableau de n entiers triés par ordre croissant.

Écrivez une fonction qui vérifie si une valeur V donnée existe dans ce tableau à l'aide de la recherche binaire, puis indiquez sa complexité.